AIOps Platform
End-to-End Project Documentation

Part 1

Professional Documentation

Prepared for GitHub Portfolio

Table of Contents

1. Executive Summary

This project is a locally deployed AIOps platform built on Kubernetes (Kind). It combines observability, incident investigation, retrieval augmented generation (RAG), local LLM reasoning, GitOps-based remediation, and an SRE-facing dashboard. The platform demonstrates how AI can assist engineers from alert detection through root-cause analysis and remediation.

2. Business Problem

Traditional Kubernetes operations require engineers to manually correlate alerts, logs, metrics, Kubernetes events, and runbooks. This project automates much of that investigation by collecting evidence, retrieving relevant knowledge, generating AI-assisted RCA, and preparing safe remediation.

3. Project Objectives

• Detect infrastructure and application issues
• Collect metrics, logs and Kubernetes events
• Perform AI-assisted root cause analysis
• Retrieve operational runbooks using RAG
• Generate investigation reports
• Support GitOps remediation workflow
• Provide an SRE-friendly dashboard

4. Technology Stack

Infrastructure: Windows 11, Docker Desktop, Kind
Monitoring: Prometheus, Alertmanager, Grafana
Logging: Loki, Promtail
AI: Ollama, SentenceTransformers
Vector DB: Qdrant
Language: Python
Deployment: Kubernetes, Helm
GitOps: GitHub, Argo CD

5. High-Level Architecture Overview

Applications
  |
Metrics / Logs / Events
  |
Prometheus + Loki
  |
Alertmanager
  |
Incident Collector
  |
Investigator
  |
RAG (Runbooks → Embeddings → Qdrant)
  |
Prompt Builder + Ollama
  |
RCA / Validation / Remediation
  |
GitHub PR → Argo CD
  |
Dashboard & SRE

6. Repository Overview

Major modules:
- aiops/agents/
- aiops/rag/
- aiops/runbooks/
- aiops/collector/
- aiops/investigator/
- aiops/remediation/
- aiops/dashboard/
- aiops/storage/
- aiops/api/
- monitoring/

7. Core Components

Incident Collector – receives alerts.
Investigator – gathers evidence.
RAG – retrieves matching runbooks.
RCA Agent – reasons over evidence.
Validation Agent – checks remediation.

Remediation Agent – prepares safe actions.
Report Agent – produces investigation report.

8. Execution Lifecycle (Preview)

1. Application issue
2. Metrics & logs generated
3. Prometheus evaluates rules
4. Alertmanager forwards alert
5. Incident Collector enriches incident
6. Investigator gathers evidence
7. RAG retrieves knowledge
8. Ollama assists RCA
9. Validation checks recommendation
10. GitOps prepares remediation
11. Dashboard presents results

Part 2 Preview

The next part will add annotated screenshots, detailed architecture diagrams, repository paths, agent workflow, GitOps remediation flow, dashboard walkthrough, and complete end-to-end execution.

AIOps Platform End-to-End Documentation

Part 2

9. Detailed Architecture

Logical Layers
1. Infrastructure (Windows, Docker, Kind)
2. Kubernetes (Applications + Monitoring)
3. Observability (Prometheus, Loki, Grafana, Alertmanager)
4. Incident Processing (Incident Collector, Investigator)
5. AI Layer (RAG, Qdrant, Ollama, Agents)
6. GitOps (Validation, Remediation, GitHub, Argo CD)
7. Presentation (Dashboard, API, SRE Copilot)

10. Incident Collection Flow

Alertmanager
↓
Incident Collector
↓
Normalize Alert
↓
Collect Metrics
Collect Logs
Collect Events
↓
Store Incident
↓
Start Investigation

Repository Paths

aiops/collector/
aiops/investigator/
monitoring/
aiops/storage/

11. RAG Pipeline

Runbooks → SentenceTransformer Embeddings → Qdrant → Similarity Search → Prompt Builder → Ollama → RCA Agent

Repository Paths

aiops/rag/ingest.py
aiops/rag/embedding.py
aiops/rag/qdrant_manager.py
aiops/rag/query.py
aiops/runbooks/

12. Agent Workflow

Assessment
↓
Investigator
↓
Evidence Summary
↓
RCA Agent
↓
Validation Agent
↓
Remediation Agent
↓
Report Agent

13. GitOps Flow

Remediation Recommendation
↓
Patch Generation
↓
GitHub Pull Request
↓
Approval
↓
Argo CD Sync
↓
Deployment Verification

Repository Paths

aiops/remediation/
aiops/policies/
aiops/api/
dashboard/
.github/

14. Screenshot Mapping

Use uploaded screenshots in final document:

- GitHub Pull Request
- Argo CD pods
- GitOps validation response
- Swagger API
- Dashboard

Each screenshot will be annotated with numbered callouts in the final merged document.

Part 3 Preview

Part 3 will merge screenshots, polished architecture diagrams, repository map, end-to-end walkthrough, and produce the final combined DOCX and PDF.